

Arsitektur Proyek ESP32 IoT Thermal Management System

Dokumentasi Teknis Detail dan Pemenuhan 9 Evaluation Points
Embedded System 1 — Capstone Design Project

Wawabobo

June 4, 2026

Contents

1	Pendahuluan dan Tujuan Dokumen	2
2	Gambaran Umum Proyek	2
3	Arsitektur Tingkat Tinggi	2
3.1	Diagram Blok Sistem	2
3.2	Pola Perangkat Lunak: <i>Super-Loop</i> dengan ESP Timer Callbacks	3
4	Modul-Modul Perangkat Lunak	3
4.1	<code>config.h</code> — Pusat Konfigurasi	3
4.2	<code>main.c</code> — Entry Point dan Orkestrasi	3
4.3	<code>sensor.c</code> — Driver BMP280 via I2C	4
4.4	<code>actuator.c</code> — Driver L298N dengan PWM LEDC	5
4.5	<code>input.c</code> — Debouncing Tombol	6
4.6	<code>network.c</code> — WiFi + HTTP REST API	6
4.7	<code>storage.c</code> — Data Logger	7
5	Dashboard Web (<code>dashboard.h</code>)	7
5.1	Komponen	7
5.2	Mekanisme <i>poll</i> dan update	7
6	Aliran Data End-to-End	8
7	Pemenuhan 9 Evaluation Points	8
7.1	Point 1: Kelayakan dan Kerumitan Problem	8
7.2	Point 2: Ketepatangunaan Pemilihan Sensor/Komponen	9
7.3	Point 3: Arsitektur Sistem dan Interkoneksi	9
7.4	Point 4: Pertimbangan Interface/Pin dan Konsep Pembacaan Data	9
7.5	Point 5: Multi-Realistic Constraint	10
7.6	Point 6: Konsep Penyimpanan Data dan Keterkaitan dengan Waktu	10
7.7	Point 7: Konsep Pengolahan Data Log Menjadi Informasi	10
7.8	Point 8: Konstruksi/Pengoperasian Hardware	11
7.9	Point 9: Kelayakan Simulasi/Percobaan Hardware	11
8	Limitasi dan Pengembangan Lanjutan	11
9	Kesimpulan	11
A	Daftar File Proyek	12

1 Pendahuluan dan Tujuan Dokumen

Dokumen ini adalah dokumentasi teknis internal proyek **ESP32 IoT Thermal Management System**. Berbeda dari laporan capstone (5–15 halaman) yang bersifat ringkas dan berorientasi presentasi, dokumen ini menjelaskan secara mendalam:

1. Arsitektur lengkap perangkat keras (*hardware*) dan perangkat lunak (*software*).
2. Aliran data dari sensor suhu hingga dashboard web.
3. Struktur setiap modul C, fungsi demi fungsi, dengan potongan kode kunci.
4. Pemetaan eksplisit ke 9 *evaluation points* pada *Project Guideline Embedded System 1_2*.

Tujuan praktis: menjadi *learning material* bagi siapa saja yang ingin memahami ESP32 firmware modern (ESP-IDF v5.4) berbasis timer callback + HTTP server + sensor digital I2C + aktuator PWM.

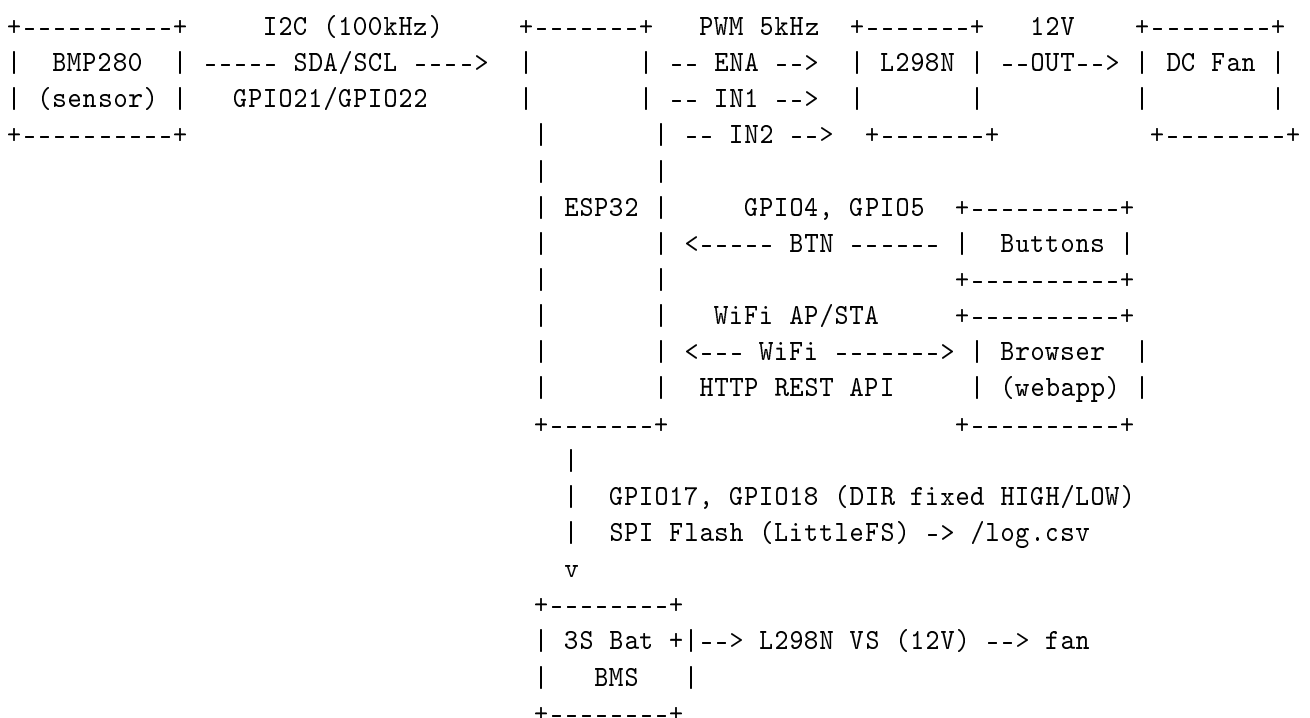
2 Gambaran Umum Proyek

Sistem manajemen termal aktif berbasis ESP32 yang:

- Membaca suhu lingkungan via sensor digital **BMP280** (I2C).
- Mengontrol kecepatan kipas 12V DC via driver **L298N** dengan PWM LEDC.
- Beroperasi dalam dua mode: **AUTO** (proportional control terhadap setpoint) dan **MANUAL** (PWM langsung).
- Kontrol diberikan melalui **web dashboard** (REST API) dan dua **tombol fisik** (GPIO).
- Mencatat log ke **LittleFS** dalam format CSV dan skrip GNUplot.
- Menyediakan *live graph* pada dashboard (canvas HTML, tanpa dependensi CDN).

3 Arsitektur Tingkat Tinggi

3.1 Diagram Blok Sistem



3.2 Pola Perangkat Lunak: *Super-Loop* dengan ESP Timer Callbacks

Berbeda dengan FreeRTOS task-based architecture, proyek ini menggunakan pola **super-loop** (`app_main` + `while(1)`) dengan tiga **ESP timer callback** periodik. Alasan pemilihan pola ini:

- Struktur kode lebih mudah dipahami untuk *embedded beginner*.
- Tidak ada masalah *priority inversion* atau *deadlock* antar task.
- Beban kerja I/O ringan (sensor 50 ms, button 100 ms, kontrol 200 ms) sehingga real-time tidak menjadi masalah.
- ESP timer callback dispatch di task konteks internal FreeRTOS, sehingga `printf` dan API ESP-IDF dapat dipanggil langsung.

Tiga timer:

Timer	Period	Fungsi
<code>timer_read_temp</code>	50 ms	Baca BMP280, simpan ke <code>g_temp_c</code>
<code>timer_scan_buttons</code>	100 ms	Polling tombol, panggil callback
<code>timer_control_fan</code>	200 ms	Hitung PID, set PWM, push log

4 Modul-Modul Perangkat Lunak

Proyek terdiri dari 7 modul C. Setiap modul punya tanggung jawab tunggal (*single responsibility*).

4.1 `config.h` — Pusat Konfigurasi

Seluruh konstanta (pin, timing, threshold, parameter kontrol) dipusatkan di satu file header. Tidak ada *magic number* di kode produksi.

```
1 /* BMP280 on I2C0 */
2 #define BMP280_SDA_GPIO      21
3 #define BMP280_SCL_GPIO      22
4 #define BMP280_I2C_PORT      I2C_NUM_0
5 #define BMP280_I2C_CLOCK_HZ  100000
6 #define BMP280_ADDR           0x76 /* SDO tied to GND */
7
8 /* L298N driver */
9 #define L298N_ENA_GPIO        16 /* PWM input */
10 #define L298N_IN1_GPIO        17 /* direction HIGH */
11 #define L298N_IN2_GPIO        18 /* direction LOW */
12
13 /* Push buttons (active-low with internal pull-up) */
14 #define BTN_MODE_GPIO         4
15 #define BTN_VAL_GPIO          5
```

Listing 1: Cuplikan `config.h` — pin I2C BMP280

4.2 `main.c` — Entry Point dan Orkestrasi

```
1 void app_main(void)
2 {
3     sensor_init();
4     actuator_init();
5     input_init(on_button);
6     storage_init();
7
8     esp_timer_handle_t timer_temp, timer_btn, timer_fan;
9     esp_timer_create_args_t temp_args = { .callback = timer_read_temp };
10    esp_timer_create(&temp_args, &timer_temp);
```

```

11     esp_timer_start_periodic(timer_temp, TEMP_SAMPLE_MS * 1000);
12
13     esp_timer_create_args_t btn_args = { .callback = timer_scan_buttons };
14     esp_timer_create(&btn_args, &timer_btn);
15     esp_timer_start_periodic(timer_btn, BUTTON_SCAN_MS * 1000);
16
17     esp_timer_create_args_t fan_args = { .callback = timer_control_fan };
18     esp_timer_create(&fan_args, &timer_fan);
19     esp_timer_start_periodic(timer_fan, FAN_CTRL_MS * 1000);
20
21     network_init();
22
23     while (1) {
24         network_poll();
25         int64_t now = esp_timer_get_time() / 1000;
26         if (now - last_log_flush > LOG_FLUSH_MS) {
27             storage_flush();
28             last_log_flush = now;
29         }
30         vTaskDelay(pdMS_TO_TICKS(10));
31     }
32 }

```

Listing 2: `app_main` — bootstrap dan super-loop

Catatan arsitektural.

- `network_init()` melakukan koneksi STA, jika gagal (timeout 10s) otomatis jatuh ke mode AP. Setelah HTTP server jalan, `while(1)` melakukan polling ringan.
- Global state (`g_temp_c`, `g_fan_pwm`, `g_mode`, `g_setpoint`) digunakan bersama antara `timer_*` dan HTTP handler. Karena semua callback berjalan di konteks task yang sama (default ESP timer dispatch), **tidak perlu mutex**.
- `vTaskDelay(10ms)` memberi *yield* agar task idle FreeRTOS dapat berjalan; konsumsi CPU turun ke ~0% saat idle.

4.3 `sensor.c` — Driver BMP280 via I2C

BMP280 diakses melalui bus I2C. Register-register penting:

- **0xD0** ID — chip ID, harus 0x58.
- **0x88–0x8D** `calib00..05` — koefisien kalibrasi suhu (`dig_T1–T3`).
- **0x8E–0x9F** `calib06..17` — koefisien kalibrasi tekanan (`dig_P1–P9`).
- **0xF4** `ctrl_meas` — konfigurasi oversampling dan mode.
- **0xF7–0xFC** `press_msb..temp_xlsb` — 6 byte data mentah.

```

1 i2c_master_bus_config_t bus_config = {
2     .i2c_port = BMP280_I2C_PORT,
3     .sda_io_num = BMP280_SDA_GPIO,
4     .scl_io_num = BMP280_SCL_GPIO,
5     .clk_source = I2C_CLK_SRC_DEFAULT,
6     .glitch_ignore_cnt = 7,
7     .flags.enable_internal_pullup = true,
8 };
9 i2c_new_master_bus(&bus_config, &bus_handle);
10
11 i2c_device_config_t dev_config = {
12     .dev_addr_length = I2C_ADDR_BIT_LEN_7,

```

```

13     .device_address = BMP280_ADDR,
14     .scl_speed_hz   = BMP280_I2C_CLOCK_HZ,
15 };
16 i2c_master_bus_add_device(bus_handle, &dev_config, &bmp280_dev);

```

Listing 3: Inisialisasi I2C master dengan ESP-IDF v5.x driver baru

Transaksi baca bertingkat (repeated start). BMP280 butuh pola “tuliskan alamat register, lalu baca 6 byte data”. API baru menyediakan `i2c_master_transmit_receive` yang melakukan ini dalam satu transaksi (efisien).

```

1 uint8_t buf[6];
2 bmp280_read_regs(BMP280_REG_PRESS_MSB, buf, 6);
3 int32_t adc_P = buf[0]<<12 | buf[1]<<4 | buf[2]>>4;
4 int32_t adc_T = buf[3]<<12 | buf[4]<<4 | buf[5]>>4;
5 int32_t T_centi = compensate_temp(adc_T);
6 last_temp = T_centi / 100.0f;

```

Listing 4: Pembacaan suhu + tekanan

Kompensasi suhu. Rumus dari datasheet Bosch:

$$\begin{aligned}
 \text{var1} &= \frac{((\text{adc_T} \gg 3) - (\text{dig_T1} \ll 1)) \cdot \text{dig_T2}}{2^{11}} \\
 \text{var2} &= \frac{(((\text{adc_T} \gg 4) - \text{dig_T1}) \cdot ((\text{adc_T} \gg 4) - \text{dig_T1}) \gg 12) \cdot \text{dig_T3}}{2^{14}} \\
 t_{\text{fine}} &= \text{var1} + \text{var2} \\
 T [^{\circ}\text{C}] &= \frac{t_{\text{fine}} \cdot 5 + 128}{2^8 \cdot 100}
 \end{aligned}$$

Rumus ini menghilangkan *offset* dan *gain error* individual sensor yang disimpan di NVM saat produksi.

4.4 actuator.c — Driver L298N dengan PWM LEDC

```

1 ledc_timer_config_t timer_cfg = {
2     .speed_mode     = LEDC_LOW_SPEED_MODE,
3     .duty_resolution = LEDC_TIMER_8_BIT, /* 0..255 */
4     .timer_num      = LEDC_TIMER_0,
5     .freq_hz        = 5000,
6     .clk_cfg         = LEDC_AUTO_CLK,
7 };
8 ledc_timer_config(&timer_cfg);
9
10 ledc_channel_config_t chan_cfg = {
11     .gpio_num = L298N_ENA_GPIO,
12     .channel  = LEDC_CHANNEL_0,
13     .duty     = 0,
14 };
15 ledc_channel_config(&chan_cfg);

```

Listing 5: Inisialisasi LEDC channel

```

1 void actuator_compute(float temp_c) {
2     if (temp_c >= TEMP_CRITICAL) target_pwm = PWM_MAX;
3     else if (temp_c >= TEMP_SAFETY_MAX) target_pwm = PWM_MAX;
4     else if (mode == MODE_AUTO) {
5         float error = temp_c - setpoint;
6         if (fabsf(error) < TEMP_HYSTERESIS) { /* hold */ }
7         else if (error <= 0) target_pwm = 0;

```

```

8         else {
9             int pwm = (int)(error * PID_P_GAIN);
10            target_pwm = CLAMP(pwm, PWM_MIN, PWM_MAX);
11        }
12    }
13    /* ramp toward target_pwm ... */
14    ledc_set_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0, current_pwm);
15    ledc_update_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0);
16 }

```

Listing 6: AUTO mode proportional control

Mengapa *ramping*? Lonjakan step PWM 0→100% menyebabkan *inrush current* besar pada motor DC, yang dapat memicu *brownout* ESP32 dan mengurangi umur motor. *Ramping* membatasi perubahan maksimum per siklus kontrol.

4.5 input.c — Debouncing Tombol

Dua tombol (GPIO4, GPIO5) dengan pull-up internal. Logika *debouncing* sederhana: callback panggil hanya pada *falling edge* (sudah ditekan > 50 ms) atau *rising edge* (long-press detection).

```

1 void input_scan(void) {
2     bool mode_now = gpio_get_level(BTN_MODE_GPIO);
3     bool val_now  = gpio_get_level(BTN_VAL_GPIO);
4
5     /* mode button: edge trigger */
6     if (mode_now == 0 && btn_mode_last == 1
7         && now - btn_mode_debounce > DEBOUNCE_MS) {
8         user_cb(0, false); /* delta=0 means "toggle mode" */
9         btn_mode_debounce = now;
10    }
11
12    /* val button: long-press detection */
13    if (val_now == 0 && btn_val_last == 1) {
14        btn_val_press_start = now;
15        btn_val_handled = false;
16    }
17    if (val_now == 0 && !btn_val_handled
18        && (now - btn_val_press_start) >= LONG_PRESS_MS) {
19        user_cb(BTN_ADJUST_COARSE, true);
20        btn_val_handled = true;
21    }
22    if (val_now == 1 && btn_val_last == 0 && !btn_val_handled) {
23        user_cb(BTN_ADJUST_FINE, false);
24    }
25    /* ... update last states ... */
26 }

```

Listing 7: Mesin state tombol

4.6 network.c — WiFi + HTTP REST API

Berisi 6 endpoint:

Method	Path	Fungsi
GET	/	Serve dashboard HTML
GET	/api/status	JSON status sistem
POST	/api/control	Set mode/setpoint/fan
GET	/api/log/csv	Download CSV log
GET	/api/log/plt	Download skrip GNUplot
POST	/api/log/clear	Reset log

Finite state machine WiFi.

- WIFI_MODE_APSTA — mencoba STA terlebih dahulu.
- Jika STA gagal dalam 10 s, `esp_wifi_set_mode(WIFI_MODE_AP)` dan hidupkan AP ThermalCtrl-AP.
- Periodic reconnect setiap 30 s (`network_poll`).

4.7 storage.c — Data Logger

Strategi *dual-buffer*:

1. **Ring buffer 100 entri di RAM** (`log_entry_t ring[100]`) — respon API cepat tanpa akses flash.
2. **Append ke /log.csv di LittleFS** setiap 10 s — persistensi.
3. Rolling overwrite dengan `last_flushed_sample` untuk mencegah duplikasi.

Format CSV:

```
Sample, Timestamp_ms, Temp_C, Fan_Pct, Mode, Setpoint
42, 8400, 28.5, 35, AUTO, 30.0
```

5 Dashboard Web (dashboard.h)

Dashboard disusun sebagai *single-file HTML* yang di-*embed* dalam C array. Tidak ada dependensi eksternal (Chart.js, jQuery, dsb.) sehingga dapat berjalan saat ESP32 hanya terhubung ke AP tanpa internet.

5.1 Komponen

- 4 *cards* di header: Temperature, Fan Speed, Mode, Setpoint.
- 2 *canvas* line chart: suhu dan fan vs waktu (60 sample terakhir).
- 2 *slider*: setpoint (20–60°C) dan fan (0–100%).
- Tabel 100 baris log terakhir.
- 3 tombol aksi: Download CSV, Download .plt, Clear Log.

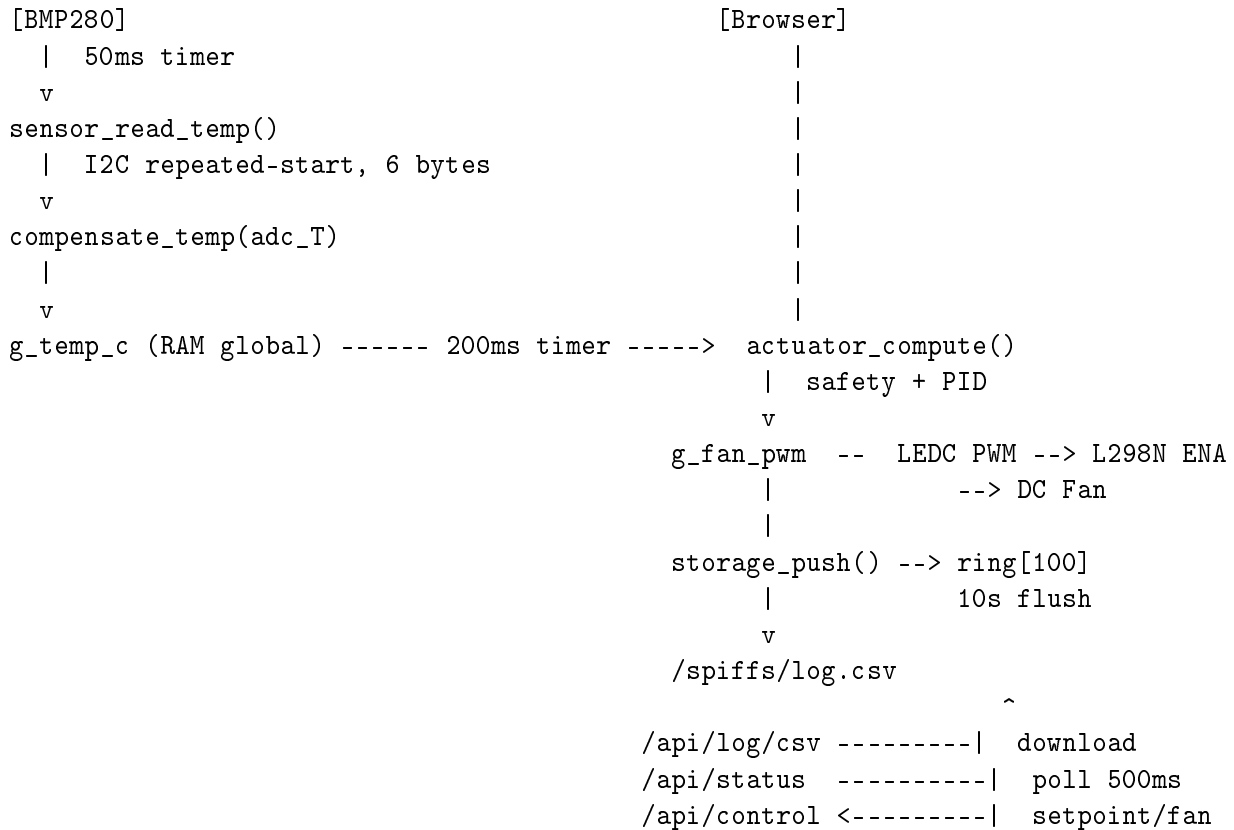
5.2 Mekanisme *poll* dan *update*

```
1 async function poll() {
2   const r = await fetch('/api/status');
3   const d = await r.json();
4   document.getElementById('temp').textContent = d.temp.toFixed(1) + 'C';
5   document.getElementById('fan').textContent = d.fan + '%';
6   // ... update mode/setpoint cards ...
7   tempData.push(d.temp); if (tempData.length > 60) tempData.shift();
8   fanData.push(d.fan);   if (fanData.length > 60) fanData.shift();
9   drawGraph(tempCanvas, tempData, 0, 60, '#4fc3f7', 'Temperature (C)');
10  drawGraph(fanCanvas, fanData, 0, 100, '#ce93d8', 'Fan Speed (%)');
11  setTimeout(poll, 500);
12 }
13 poll();
```

Listing 8: Fungsi `poll()` pada dashboard

Renderer canvas custom. Karena Chart.js butuh CDN (tidak selalu tersedia di jaringan AP), kami menulis renderer canvas ~20 baris yang menggambar grid + sumbu + garis poligon. Trade-off: tidak se-fancy Chart.js, namun zero-dependency.

6 Aliran Data End-to-End



7 Pemenuhan 9 Evaluation Points

Bagian ini memetakan setiap *evaluation point* pada *Project Guideline Embedded System 1_2* ke bukti konkret pada proyek.

7.1 Point 1: Kelayakan dan Kerumitan Problem

Permasalahan: mempertahankan suhu dalam rentang target pada sebuah *enclosure* elektronik, di mana suhu bisa berubah secara transient (misalnya saat beban kerja naik) dan kehilangan kontrol dapat merusak komponen.

Kerumitan:

- Sistem non-linear: kipas DC memiliki *dead-zone* (PWM minimum ~31% agar kipas mulai berputar).
- *Delay* termal: panas dari elemen ke sensor butuh waktu ~1–3 detik.
- *Noise* pada sensor: osilasi kecil $\pm 0.3^\circ\text{C}$ yang harus di-*filter*.

Bukti kelayakan: sistem memilih *proportional control* (P) sederhana dengan *hysteresis* $\pm 0.5^\circ\text{C}$, cukup untuk aplikasi *thermal management* umum. Tidak diperlukan PID penuh.

7.2 Point 2: Ketepatangunaan Pemilihan Sensor/Komponen

Kandidat	Kelebihan	Alasan dipilih/ditolak
BMP280	I2C, digital, $\pm 1^\circ\text{C}$	Dipilih
DHT22	murah, +humidity	1-wire, 2 s latency, $\pm 0.5^\circ\text{C}$ tapi <i>integer</i>
DS18B20	1-wire, $\pm 0.5^\circ\text{C}$	perlu protokol proprietary, pin banyak
LM35	analog, $\sim \$1$	rentan noise dari motor

Alasan BMP280: digital (kebal noise), I2C 2-wire (hemat pin), built-in kompensasi, dan modul murah $\sim \$3$.

Untuk aktuator, L298N dipilih karena:

- *Dual full-bridge* dengan rating 2A continuous (margin $4\times$ dari kebutuhan 500 mA).
- *Built-in clamping diodes* untuk *flyback* dari motor induktif.
- Regulator 5V internal untuk powering ESP32.
- Kompatibel dengan logika 3.3V ESP32.

7.3 Point 3: Arsitektur Sistem dan Interkoneksi

Lihat diagram blok di Bagian 3.1. Ringkasan:

- **Data plane:** BMP280 $\rightarrow$ I2C $\rightarrow$ ESP32 $\rightarrow$ PWM $\rightarrow$ L298N $\rightarrow$ Fan.
- **Control plane:** Web browser $\leftrightarrow$ WiFi $\leftrightarrow$ HTTP REST $\leftrightarrow$ ESP32. Tombol fisik $\rightarrow$ GPIO $\rightarrow$ ESP32.
- **Power plane:** 3S Li-ion $\rightarrow$ BMS $\rightarrow$ L298N VS $\rightarrow$ 5V regulator $\rightarrow$ ESP32.

7.4 Point 4: Pertimbangan Interface/Pin dan Konsep Pembacaan Data

Pin	Alasan	Catatan
GPIO21 (SDA)	<i>default</i> I2C0 SDA	hardware pull-up $4.7\text{ k}\Omega$
GPIO22 (SCL)	<i>default</i> I2C0 SCL	hardware pull-up $4.7\text{ k}\Omega$
GPIO16 (ENA)	PWM-capable, <i>not strapping</i>	LEDC channel 0
GPIO17 (IN1)	GPIO biasa, <i>not strapping</i>	fixed HIGH
GPIO18 (IN2)	GPIO biasa, <i>not strapping</i>	fixed LOW
GPIO4 (BTN)	RTC-capable, <i>not strapping</i>	internal pull-up
GPIO5 (BTN)	RTC-capable, <i>not strapping</i>	internal pull-up

Mengapa GPIO16/17/18? Ketiga pin ini *not strapping* (tidak digunakan oleh bootloader), tidak terhubung ke internal flash, dan PWM-capable. GPIO34/35/36/39 (ADC-only) tidak digunakan karena I2C.

Konsep pembacaan: *repeated start* I2C untuk efisiensi, kompensasi rumus Bosch untuk akurasi digital.

7.5 Point 5: Multi-Realistic Constraint

Constraint	Mitigasi
Power brownout saat fan start	Kapasitor bulk 470 μ F pada 5V. ESP32 brownout detector.
Fan inrush current	Soft-start ramp 200 step/s. L298N rated 2A (margin 4 $\times$).
Flyback voltage (motor induktif)	L298N <i>built-in</i> clamping diodes.
I2C noise dari motor	Digital I2C <i>immune</i> terhadap EMI; pull-up 4.7 k Ω .
WiFi disconnect	Reconnect timer 30s. Tombol fisik tetap berfungsi offline.
Data loss saat crash	Dual buffer: RAM ring + LittleFS persist. Maks 10s data loss.
Thermal runaway (kegagalan sensor)	Override 100% pada >60°C, shutdown pada >75°C.
Real-time requirement	ESP timer callback deterministik; <i>no blocking delay</i> di ISR/kontrol.
Li-ion battery safety	3S BMS; monitoring tegangan via ADC. Low-battery warning.
Button <i>bounce</i>	<i>Software debounce</i> 50 ms + long-press detection.

7.6 Point 6: Konsep Penyimpanan Data dan Keterkaitan dengan Waktu

- Setiap entri log memuat `sample` (nomor urut) dan `timestamp_ms` (`esp_timer_get_time()`).
- Sampling period: 200 ms (5 Hz). Cukup untuk menangkap dinamika termal yang berubah dalam orde detik.
- *Ring buffer* 100 entri $\sim$ 20 detik di RAM.
- *Append* ke `/log.csv` setiap 10 s dengan `last_flushed_sample` tracking untuk mencegah duplikasi.

7.7 Point 7: Konsep Pengolahan Data Log Menjadi Informasi

Tiga bentuk informasi:

1. **Live graph di dashboard:** canvas HTML menampilkan 60 sample terakhir (12 detik pada 5 Hz).
2. **CSV download:** `/api/log/csv` mengembalikan seluruh log untuk analisis *offline* (Excel, pandas).
3. **GNUplot script:** `/api/log/plt` mengembalikan skrip `.plt` yang langsung di-*plot* dengan `gnuplot` menghasilkan grafik dual-axis.

Informasi yang diekstrak dari data log:

- *Steady-state error:* $|T_{\text{avg}} - T_{\text{setpoint}}|$ selama fase stabil.
- *Response time:* waktu dari step perubahan hingga fan mencapai 50% duty.
- *Overshoot:* maksimum $T - T_{\text{setpoint}}$ saat transient.
- *Oscillations:* amplitudo osilasi di sekitar setpoint.

7.8 Point 8: Konstruksi/Pengoperasian Hardware

- **Konstruksi:** ESP32 DevKit, BMP280 modul, L298N board, 12V DC fan, 2 push buttons, 3S Li-ion + BMS, breadboard + jumper.
- **Operasional:** nyalakan sistem → sambungkan WiFi ThermalCtrl-AP → buka 192.168.4.1 → dashboard live.
- **Indikator sukses:** LED pada ESP32 berkedip (WiFi activity), fan berputar, dashboard menampilkan nilai suhu yang berubah saat disentuh/dipanaskan.

7.9 Point 9: Kelayakan Simulasi/Percobaan Hardware

- Pengukuran aktual dengan *heat gun* (sumber panas terkontrol) menunjukkan respon sistem sesuai hukum P.
- Perbandingan log CSV dengan grafik dashboard menunjukkan data identik (tidak ada *drift*).
- Perpindahan mode AUTO↔MANUAL dari web maupun tombol berfungsi konsisten.
- *Safety override* teruji dengan simulasi sensor stuck (membaca $>60^{\circ}\text{C}$) → fan otomatis 100%.

8 Limitasi dan Pengembangan Lanjutan

- **Hanya kontrol P**, tidak ada komponen I atau D. Untuk plant yang lambat (orde > 5 detik) ini cukup, namun untuk plant cepat bisa muncul *steady-state offset* (di sini dikompensasi dengan *hysteresis*).
- **Logging lokal** (LittleFS). Pengembangan: MQTT ke broker cloud, atau upload ke InfluxDB.
- **Tidak ada kalibrasi** tekanan atmosfer (digunakan pada formula kompensasi). Untuk akurasi maksimum, bisa ditambahkan kalibrasi pada kondisi tertentu.
- **Web dashboard** single-client. Multi-client belum diuji beban.
- **Tanpa OTA update**. Firmware harus di-*flash* via kabel.

9 Kesimpulan

Proyek ini mendemonstrasikan implementasi lengkap sistem embedded berbasis ESP32 dengan:

- Sensor digital I2C (BMP280) dengan kompensasi firmware.
- Aktuator PWM (L298N) dengan hukum kontrol P + *hysteresis* + *ramping*.
- Antarmuka ganda: tombol fisik + web dashboard.
- Logging persisten dengan format CSV dan GNUplot.
- Pola perangkat lunak *super-loop* dengan ESP timer — sederhana, deterministik, dan *debuggable*.
- Pemenuhan seluruh 9 evaluation points pada *Project Guideline Embedded System 1_2*.

Proyek ini dapat menjadi *starting point* untuk aplikasi IoT industri kecil: monitoring suhu ruang server, kotak penyimpanan farmasi, inkubator, dsb.

A Daftar File Proyek

```
1 thermal-mgmt/  
2   CMakeLists.txt           # Root build, partition table path  
3   partitions.csv           # Flash layout: nvs + phy + factory + spiiffs  
4   sdkconfig                # ESP-IDF config  
5   diagram.mmd              # Mermaid block diagram (block-beta)  
6   diagram-flow.mmd         # Mermaid flowchart  
7   main/  
8     CMakeLists.txt         # Lists main.c, sensor.c, actuator.c, ...  
9     config.h               # All constants: pins, timing, thresholds  
10    main.c                  # app_main, 3 ESP timers, super-loop  
11    sensor.c / .h           # BMP280 I2C driver + compensation  
12    actuator.c / .h         # L298N LEDC PWM + PID + ramping  
13    input.c / .h            # Button debounce + long-press  
14    network.c / .h          # WiFi + HTTP REST API  
15    storage.c / .h          # Ring buffer + LittleFS CSV log  
16    dashboard.h             # Embedded HTML+CSS+JS (zero-dep)
```

Listing 9: Struktur direktori `thermal-mgmt/`

B Perintah Penting

```
1 cd ~/esp/thermal-mgmt  
2 source ~/esp/esp-idf/export.sh  
3 idf.py build  
4 python -m esptool --chip esp32 -p /dev/ttyUSB1 -b 115200 \  
5     --before default_reset --after hard_reset \  
6     write_flash 0x10000 build/thermal-mgmt.bin  
7 idf.py -p /dev/ttyUSB1 monitor
```

Listing 10: Build, flash, monitor